

	Departamento: Ciencias de la Computación
	Carrera: Electrónica y Automatización Asignatura: Fundamentos de Programación NRC: 28561
	Apellidos y nombres del estudiante: Figueroa Cerón Sofía Vanessa

Taller U3 Funcion

DESARROLLO

Ejercicio 56: Funciones void, variables globales y bandera

Objetivo: Comprender funciones void, variables globales y uso de una variable bandera para decidir el mensaje de salida.

Conceptos aplicados: ingresar(), imprimir(), multiplicar(), dividir(), variable band, validación $n2 \neq 0$.

Instrucciones específicas:

1. Ejecutar el programa e ingresar dos números enteros.
2. Observar que multiplicar() calcula $n3 = n1 * n2$ y asigna $band = 1$.
3. Observar que dividir() valida si $n2$ es diferente de cero. Si $n2$ es cero, asigna $band = 3$.
4. Analizar cómo imprimir() cambia el mensaje según el valor de band.
5. Modificar el programa para agregar una función sumar() y una función restar() usando la misma lógica de bandera.

Pruebas mínimas	Resultado esperado por el estudiante
$n1=8, n2=2$	Registrar salida de pantalla y explicar el flujo de funciones.
$n1=10, n2=0$	Registrar salida de pantalla y explicar el flujo de funciones.
$n1=-6, n2=3$	Registrar salida de pantalla y explicar el flujo de funciones.

Código

```
#include <stdio.h>
```

```
/* Variables globales */
```

```
int n1, n2;
```

```
int n3;
```

```
float resultadoDivision;
```

```
int band;
```

```
/* Funcion para ingresar los numeros */
```

```
void ingresar()
```

```
{
```

```
    printf("Ingrese el primer numero: ");
```

```
    scanf("%d", &n1);
```

```
    printf("Ingrese el segundo numero: ");
```

```
    scanf("%d", &n2);
```

```
}
```

```
/* Funcion para multiplicar */
```

```
void multiplicar()
```

```
{
```

```
    n3 = n1 * n2;
```

```
    band = 1;
```

```
}
```

```
/* Funcion para dividir */
```

```
void dividir()
```

```
{
```

```
    if (n2 != 0)
```

```
    {
```

```
        resultadoDivision = (float)n1 / n2;
```

```
        band = 2;
```

```
}
```

```
    else

    {
        band = 3;
    }
}

/* Funcion para sumar */
void sumar()
{
    n3 = n1 + n2;

    band = 4;
}

/* Funcion para restar */
void restar()
{
    n3 = n1 - n2;

    band = 5;
}

/* Funcion para imprimir resultados segun la bandera */
void imprimir()
{
    if (band == 1)
    {
        printf("Resultado de la multiplicacion: %d\n", n3);
    }

    else if (band == 2)
    {
```

```

    printf("Resultado de la division: %.2f\n", resultadoDivision);
}
else if (band == 3)
{
    printf("Error: no se puede dividir para cero.\n");
}
else if (band == 4)
{
    printf("Resultado de la suma: %d\n", n3);
}
else if (band == 5)
{
    printf("Resultado de la resta: %d\n", n3);
}
}

int main()
{
    printf("=== FUNCIONES VOID, VARIABLES GLOBALES Y BANDERA ===\n\n");
    ingresar();
    printf("\n--- Resultados ---\n");
    multiplicar();
    imprimir();
    dividir();
    imprimir();
    sumar();
    imprimir();
}

```

```
    restar();

    imprimir();

    return 0;

}
```

Validar Código

```
C:\Users\G300\Desktop\Unitt  X  +  v
=== FUNCIONES VOID, VARIABLES GLOBALES Y BANDERA ===

Ingrese el primer numero: 8
Ingrese el segundo numero: 2

--- Resultados ---
Resultado de la multiplicacion: 16
Resultado de la division: 4.00
Resultado de la suma: 10
Resultado de la resta: 6

Process returned 0 (0x0)   execution time : 79.546 s
Press any key to continue.
|
```

```
C:\Users\G300\Desktop\Unitt  X  +  v
=== FUNCIONES VOID, VARIABLES GLOBALES Y BANDERA ===

Ingrese el primer numero: 10
Ingrese el segundo numero: 0

--- Resultados ---
Resultado de la multiplicacion: 0
Error: no se puede dividir para cero.
Resultado de la suma: 10
Resultado de la resta: 10

Process returned 0 (0x0)   execution time : 3.370 s
Press any key to continue.
|
```

Pregunta de análisis: ¿Por qué la función imprimir() no necesita recibir parámetros en este ejercicio?

La función imprimir() no necesita recibir parámetros porque el programa está usando **variables globales**. Esto significa que variables como n1, n2, n3, resultadoDivision y band pueden ser usadas por todas las funciones del programa. Por eso, imprimir() revisa directamente el valor de band y muestra el mensaje correspondiente sin necesidad de recibir datos dentro de los paréntesis.

Ejercicio 60: División con validación

Objetivo: Crear una función de división que reciba dos números reales y valide división para cero.

Conceptos aplicados: float dividir(float a, float b), validación if (b != 0), retorno de resultado.

Instrucciones específicas:

6. Ejecutar el programa e ingresar dos números reales.
7. Probar una división válida y verificar el resultado con dos decimales.
8. Probar una división con divisor cero y observar el mensaje.
9. Analizar la limitación del programa: si el resultado real de una división es 0, podría confundirse con error.
10. Mejorar el programa usando una variable bandera o una función adicional para validar el divisor antes de dividir.

Pruebas mínimas	Resultado esperado por el estudiante
n1=8, n2=2	Registrar salida de pantalla y explicar el flujo de funciones.
n1=0, n2=5	Registrar salida de pantalla y explicar el flujo de funciones.
n1=9, n2=0	Registrar salida de pantalla y explicar el flujo de funciones.

Código

```
#include <stdio.h>

// Funcion que realiza la division y retorna el resultado

float dividir(float a, float b)

{

    return a / b;

}

// Funcion que valida si el divisor es diferente de cero
```

```

int validarDivisor(float b)
{
    if (b != 0)
    {
        return 1; // Si se puede dividir
    }
    else
    {
        return 0; // No se puede dividir
    }
}

int main()
{
    float n1, n2, resultado;

    printf("=== DIVISION CON VALIDACION ===\n");

    printf("Ingrese el primer numero: ");

    scanf("%f", &n1);

    printf("Ingrese el segundo numero: ");

    scanf("%f", &n2);

    if (validarDivisor(n2) == 1)
    {
        resultado = dividir(n1, n2);

        printf("\n--- Resultado ---\n");

        printf("La division de %.2f para %.2f es: %.2f\n", n1, n2, resultado);
    }
    else

```

```

{

printf("\n--- Resultado ---\n");

printf("Error: no se puede dividir para cero.\n");

}

return 0;

}

```

Validación Código:

The screenshot displays the Code::Blocks IDE with a C++ project named 'Untitled2.c'. The code implements a division function with validation. The terminal window shows the program's execution, where the user enters 8 and 2, resulting in a division of 4.00. The program then returns 0 and displays the execution time.

```

1  #include <stdio.h>
2  // Funcion que realiza la division y retorna el resultado
3  float dividir(float a, float b)
4  {
5      return a / b;
6  }
7  // Funcion que valida si el divisor es diferente de cero
8  int validarDivisor(float b)
9  {
10     if (b != 0)
11     {
12         return 1; // Si se puede dividir
13     }
14     else
15     {
16         return 0; // No se puede dividir
17     }
18 }
19 int main()
20 {
21     float n1, n2, resultado;
22     printf("=== DIVISION CON VALIDACION ===\n");
23     printf("Ingrese el primer numero: ");
24     scanf("%f", &n1);
25     printf("Ingrese el segundo numero: ");
26     scanf("%f", &n2);
27     if (validarDivisor(n2) == 1)
28     {
29         resultado = dividir(n1, n2);
30         printf("\n--- Resultado ---\n");
31         printf("La division de %.2f para %.2f es: %.2f\n", n1, n2, resultado);
32     }
33     else
34     {
35         printf("\n--- Resultado ---\n");
36         printf("Error: no se puede dividir para cero.\n");
37     }
38     return 0;
39 }
40

```

```

C:\Users\G300\Desktop\Unittit  x  +  v
=== DIVISION CON VALIDACION ===
Ingrese el primer numero: 8
Ingrese el segundo numero: 2

--- Resultado ---
La division de 8.00 para 2.00 es: 4.00

Process returned 0 (0x0)   execution time : 15.682 s
Press any key to continue.

```

C:\Users\G300\Desktop\Untitled2.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 39, Col 2, Pos 947 | Insert | Read/Write | default

The image shows a C++ IDE window titled 'Untitled2.c - Code::Blocks 25.03'. The code defines a function `dividir` that takes two floats and returns their division. It also defines a function `validarDivisor` that checks if the divisor is zero. The `main` function prompts the user for two numbers, calls `validarDivisor` on the second number, and then calls `dividir` if the divisor is non-zero. The terminal output shows the program running with inputs 9 and 0, resulting in an error message: 'Error: no se puede dividir para cero.' The status bar at the bottom indicates the file path, C/C++ compiler, and current line/col (Line 39, Col 2, Pos 947).

```
1 #include <stdio.h>
2 // Funcion que realiza la division y retorna el resultado
3 float dividir(float a, float b)
4 {
5     return a / b;
6 }
7 // Funcion que valida si el divisor es diferente de cero
8 int validarDivisor(float b)
9 {
10     if (b != 0)
11     {
12         return 1; // Si se puede dividir
13     }
14     else
15     {
16         return 0; // No se puede dividir
17     }
18 }
19 int main()
20 {
21     float n1, n2, resultado;
22     printf("=== DIVISION CON VALIDACION ===\n");
23     printf("Ingrese el primer numero: ");
24     scanf("%f", &n1);
25     printf("Ingrese el segundo numero: ");
26     scanf("%f", &n2);
27     if (validarDivisor(n2) == 1)
28     {
29         resultado = dividir(n1, n2);
30         printf("\n--- Resultado ---\n");
31         printf("La division de %.2f para %.2f es: %.2f\n", n1, n2, resultado);
32     }
33     else
34     {
35         printf("\n--- Resultado ---\n");
36         printf("Error: no se puede dividir para cero.\n");
37     }
38     return 0;
39 }
```

```
=== DIVISION CON VALIDACION ===
Ingrese el primer numero: 9
Ingrese el segundo numero: 0

--- Resultado ---
Error: no se puede dividir para cero.

Process returned 0 (0x0)   execution time : 5.997 s
Press any key to continue.
```

Pregunta de análisis

¿Por qué retornar 0 para indicar error puede generar confusión en una división válida?

En este programa se usa una función aparte llamada `validarDivisor()` para verificar primero si el divisor es diferente de cero, además matemáticamente no se puede dividir un número 0. Así se evita confundir un resultado válido con un error.